Concurrency and Synchronization Roadmap

- Concurrency and Synchronization
  - ➢ Mutual exclusion and critical section
  - ➢ Lost update problem and busy waiting vs. blocking
  - ➢ Interrupt disabling and spinlock
  - ➢ Semaphores and Readers/Writers problem
  - ➢ Deadlock and starvation

Operating Systems

Lost update problem & Interrupt disabling and spinlock

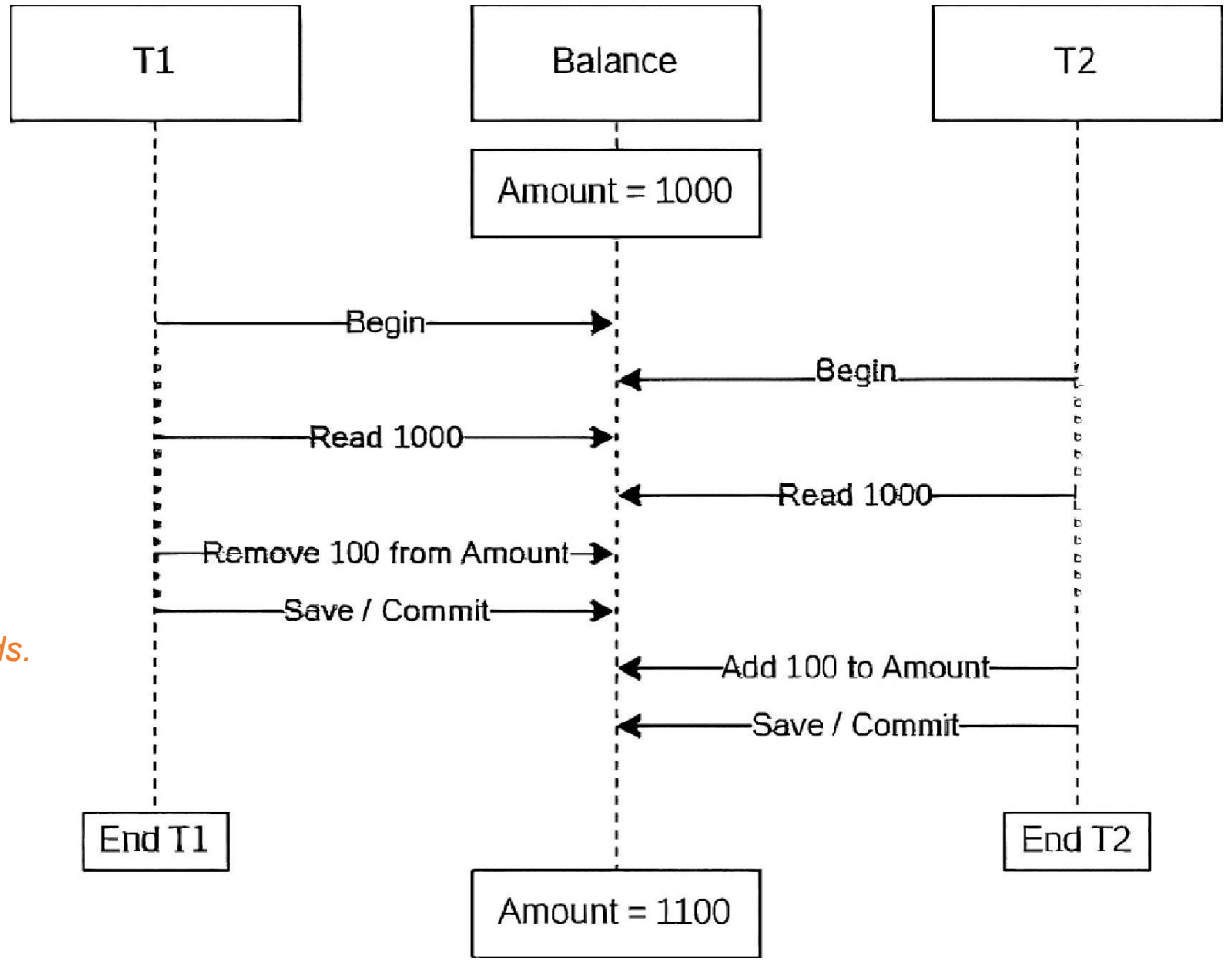
Chapter 5 — Concurrency: Mutual Exclusion and Synchronization

What Is the Lost Update Problem

The lost update problem is a concurrency issue that arises when multiple processes or threads concurrently access and modify the same data, resulting in some updates unintentionally overwriting others

. This problem is significant in any system where concurrent modifications are possible, such as collaborative applications, financial systems, or any scenario involving shared resources.

However, T1 removes some amount from the balance while T2 adds. Eventually, one of the two transactions will happen at the end, overwriting the other transaction's change. While the final amount should acknowledge both updates, one lost update will occur if no concurrency strategies exist.



What Causes the Lost Update Problem

To trigger potential lost-update risk, all of following conditions must be satisfied:

01

Multi-thread / multi-process environment

02

At least two threads perform read-modify-write with write-back to shared variable

03

At least one of these read-modify-write sequences is non-atomic

Lost updates

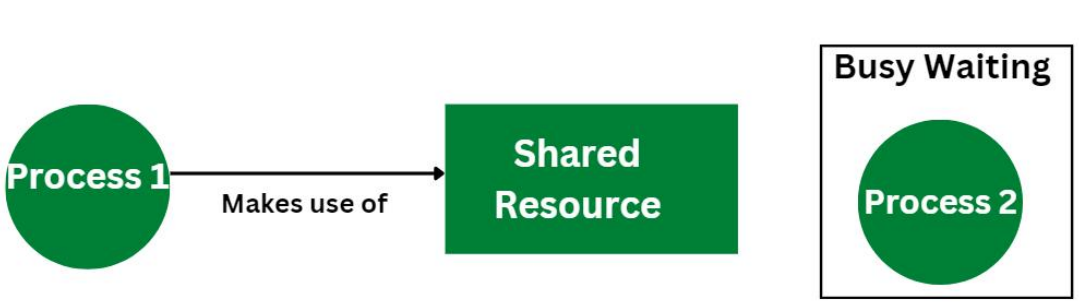
Scenario	Thread-A behaviour	Thread-B behaviour	Is read-modify-write non-atomic?	Possible Lost-Updates?	Remark (other possible problem)
1	Only read	Only read	-	No	No concurrency problem
2	Only read	Non-atomic R-M-W	Yes	No	May cause dirty read
3	Non-atomic R-M-W	Only read	Yes	No	May cause dirty read
4	Non-atomic R-M-W	Non-atomic R-M-W	Yes	Yes	Risk of lost updates
5	Atomic R-M-W	Non-atomic R-M-W	Only A is atomic	Yes	B is non-atomic, interleaving can lead to lost updates
6	Atomic R-M-W	Atomic R-M-W	-	No	Atomic operation prevents lost updates

•**Dirty read**: when we read data modified by others but not yet saved。

What is Busy Waiting?

Busy Waiting is also known as busy looping or spinning. The condition that is to be checked is the entry condition to be true

Consider a scenario where a resource is required by the process for execution of a specific program. The resource is being used by another process and is unavailable at that moment. Therefore the process needs to wait for the resource to become available.



NEED: Busy waiting is required in operating system for achieving mutual exclusion.

✕ 01. Busy waiting keeps CPU busy at all the time until it's condition gets satisfied.

✕ 02. Busy waiting consumes more power.

Busy Waiting vs Blocking Waiting

Indicators	Busy-Waiting (Spin-lock)	Blocking Waiting (Mutex)
Behaviour when lock unavailable	Hold CPU	Give-up CPU and enter blocked state
CPU consumption during wait	Wastes CPU cycles	No CPU consumption
Context-switch overhead	None	Incurs switch overhead (block / wake-up)
Priority inversion risk	Single-core: may cause unbounded priority inversion	Only bounded priority inversion
Best-case scenario	Short critical sections (multi-core)	Long critical sections



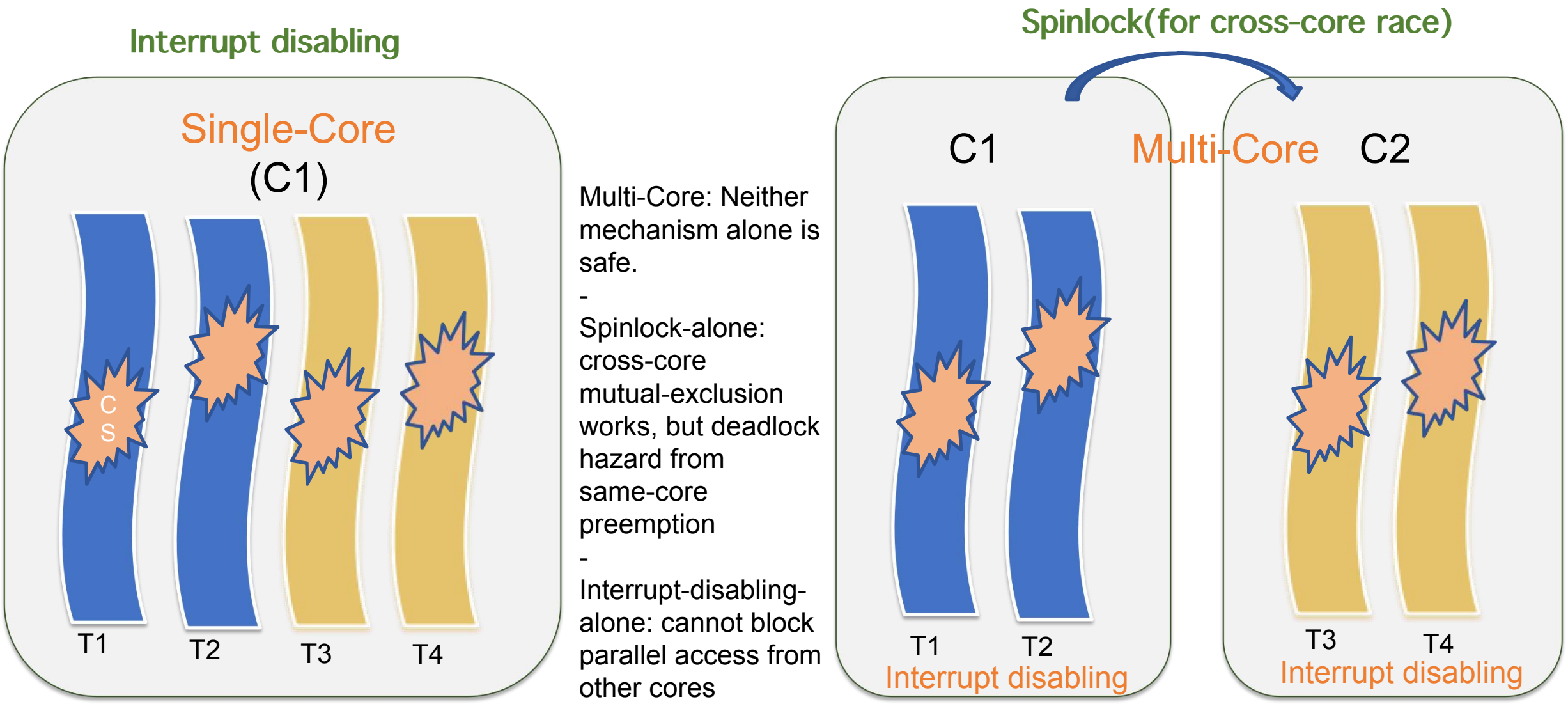
Interrupt disabling and spinlock

Why neither interrupt disabling nor spinlock alone is enough?

Two independent mechanisms, two different threats.

We need to understand what each one actually protects against.

Final solution： Interrupt disabling + Spinlock



Multi-Core: Neither mechanism alone is safe.

-

Spinlock-alone: cross-core mutual-exclusion works, but deadlock hazard from same-core preemption

-

Interrupt-disabling-alone: cannot block parallel access from other cores

No other cores, no hardware parallelism.

cores, hardware parallelism.

Conclusions

- Single-core: race comes from timer-driven preemption. Disable local IRQ provides mutual-exclusion; spinlock should not be used alone.
- Multi-core has two separate hazards:
  - Cross-core parallel race → solved by spinlock
  - Same-core preemption-caused deadlock → solved by disabling local IRQ
- Spinlock-alone: cross-core mutual-exclusion still works, yet deadlock hazard remains.
- Disable-local-IRQ-alone: suppresses local preemption, but cannot stop parallel access from other cores.
- Standard kernel solution for multi-core: Disable local IRQ + Spinlock

Where Do Race Conditions Come From?

Source 1: Same-Core Preemption

- Timer interrupt fires on the current CPU core.
- The scheduler preempts the running thread.
- Another thread on the same core is dispatched.
- Both threads may access the same shared data.

Source 2: Cross-Core Parallelism

- Multiple CPU cores execute independently at the same time.
- Threads on different cores may enter the critical section simultaneously.
- No interrupt is involved.
- This only exists on multi-core systems.

Two different threats. Each mechanism addresses only one of them.

Interrupt Disabling: What It Does and What It Does Not

How It Works

- Disable interrupts on the current CPU core.
- Timer interrupt is blocked → no preemption.
- The current thread cannot be switched out.
- It runs to completion inside the critical section.

Effective on single-core systems:
prevents same-core preemption.

What It Cannot Do

- Only affects the current CPU core.
- Other cores still execute threads freely.
- Cross-core race conditions remain unsolved.
- Interrupts must not be disabled for long → I/O delays.

Ineffective on multi-core systems:
other cores are not stopped.

Spinlock: Cross-Core Mutual Exclusion

How It Works

Spinlock uses a shared memory **flag (0 = free, 1 = held).** A thread that cannot acquire the lock spins in a tight loop, repeatedly checking the flag.
It works across CPU cores because the flag is in shared memory, and the check-and-set operation is atomic.

Why Spinlock Alone Deadlocks on Single-Core

Step 1
Thread T1 holds the spinlock, in critical section.

Step 2
Timer interrupt fires. Scheduler preempts T1, dispatches T2.

Step 3
T2 tries to acquire the same lock. Starts spinning in busy-wait loop.

Step 4: DEADLOCK
T2 never yields CPU. T1 never runs again. Lock is never released.

Spinlock solves cross-core races, but it does **not** prevent same-core preemption.
If the lock holder is preempted on the same core, the waiting thread spins forever.

The Combined Mechanism: Disable Local IRQ + Spinlock

IRQ = Interrupt Request

Entering the Critical Section

1. Disable local interrupt on current core
2. Acquire the spinlock
3. Execute the critical section

Exiting the Critical Section

1. Release the spinlock
2. Re-enable local interrupt

Order matters! Never enable interrupt before releasing the spinlock.

- Step 1: suppresses same-core concurrency caused by scheduler preemption.
- Step 2: blocks cross-core parallel access to shared data.

Takeaway: **Disable local IRQ eliminates the risk of scheduler-driven concurrency on the current core. Spinlock blocks hardware-level parallel access from other cores. Together they establish full mutual-exclusion for shared data inside the critical section.**

Key Takeaways

Mechanism	Same-Core Preemption	Cross-Core Parallelism	Safe on
Interrupt Disabling	Prevents it	Does not stop it	Single-core only
Spinlock Alone	May deadlock upon same-core preemption	Prevents it	Never alone
Combined	Prevents it	Prevents it	Multi-core

Points:

1. Disabling local interrupt only affects the current CPU core.
2. Spinlock used alone (without local IRQ disable) may cause deadlock on both single-core and multi-core systems, if same-core preemption occurs.
3. Enter critical section: disable local IRQ first, then acquire spinlock.
4. Exit critical section: release spinlock first, then re-enable local IRQ.

Exercise 1 True / False

1.Disabling local IRQ for current core can stop parallel access from other CPU cores to shared variables.

2.When using spinlock alone in kernel, deadlock may occur on multi-core system if same-core preemption happens.

3.The purpose of disabling local IRQ before acquiring spinlock is to eliminate same-core scheduler preemption risk.

4.For multi-core kernel, we can safely use spinlock without disabling local IRQ.

Exercise 2 Order puzzle

Question： We use combined mechanism: disable local IRQ + spinlock. Which sequence is correct for entering and leaving critical section?

A. Enter: Acquire spinlock → Disable local IRQ； Exit: Enable local IRQ → Release spinlock

B. Enter: Disable local IRQ → Acquire spinlock； Exit: Release spinlock → Enable local IRQ

C. Enter: Disable local IRQ → Acquire spinlock； Exit: Enable local IRQ → Release spinlock

1.What is the main purpose of disabling local IRQ before acquiring a kernel spinlock?
A. To block other CPU cores from accessing shared data
B. To eliminate the risk of same-core scheduler preemption
C. To stop all hardware interrupts of the whole system
D. To make spinlock run faster

2.If we use spinlock alone without disabling local IRQ, when can deadlock arise in kernel-level code?
A. Only on single-core systems
B. Only on multi-core systems
C. When same-core preemption takes place
D. Whenever two cores contend for this lock

3.Spinlock itself solves which race condition in multi-core kernel?
A. Same-core concurrency caused by time-slicing preemption
B. Cross-core parallel access to shared data
C. Deadlock introduced by timer interrupt
D. Starvation of low-priority kernel threads

4.Disabling local IRQ on one CPU core will:
A. Mask interrupts for all CPU cores in the system
B. Prevent other cores from entering critical section
C. Only affect interrupt handling of the current core
D. Guarantee full mutual-exclusion for multi-core kernel by itself